

**Міністерство освіти і науки України**  
**Національний університет «Запорізька політехніка»**

Кафедра програмних засобів

**ЗВІТ**

Дисципліна «Інженерія програмного забезпечення вбудованих систем»

Робота №2

Тема «Моделювання роботи ультразвукового датчика вимірювання  
відстані та сервоприводів»

**Виконав варіант 19**

Студент КНТ-122

Онищенко О.А.

**Прийняли**

Викладач

Гладкова О.М.

## МЕТА

Ознайомитись з ультразвуковим датчиком вимірювання відстані та сервоприводами, навчитися створювати схеми Arduino з використанням цих компонентів в середовищі Tinkercad.

## ЗАВДАННЯ

Завдання (з дозволу викладача лекцій Пархоменко А.В.) виконано за методичними вказівками для спеціальності 122.

Текст індивідуального завдання:

1. Створити схему, як зображено на рис. 2.4 та додати код.
2. Додайте на схему п'ять світлодіодів. Якщо відстань до об'єкта менше 50 см, то всі світлодіоди вмикаються. Якщо відстань більше 50 см і менше 100 см → 4 світлодіоди вмикаються. Якщо відстань більше 100 см і менше 150 см → 3 світлодіоди вмикаються. Якщо відстань більше 150 см і менше 200 см → 2 світлодіоди вмикаються. Якщо відстань більше 200 см і менше 300 см → один світлодіод вмикається. Якщо відстань перевищує 300 см, усі світлодіоди вимикаються.
3. Створіть схему зі світлодіодом і датчиком відстані. Чим ближче відстань до об'єкта, тим яскравіше світиться світлодіод.
4. Створіть схему з сервоприводом і ультразвуковим датчиком відстані. Положення сервопривода за замовчуванням становить 90 градусів. Якщо відстань до об'єкта менше 100 см, то сервопривід повертається на 0 градусів, якщо відстань більше 100 см – на 180 градусів.

## **ВИКОНАННЯ**

### **1 Приклад ультразвукового датчику відстані**

На область було додано плату Arduino, макетоплату. На макетоплату додано ультразвуковий датчик вимірювання дистанції та світлодіод з резистором:

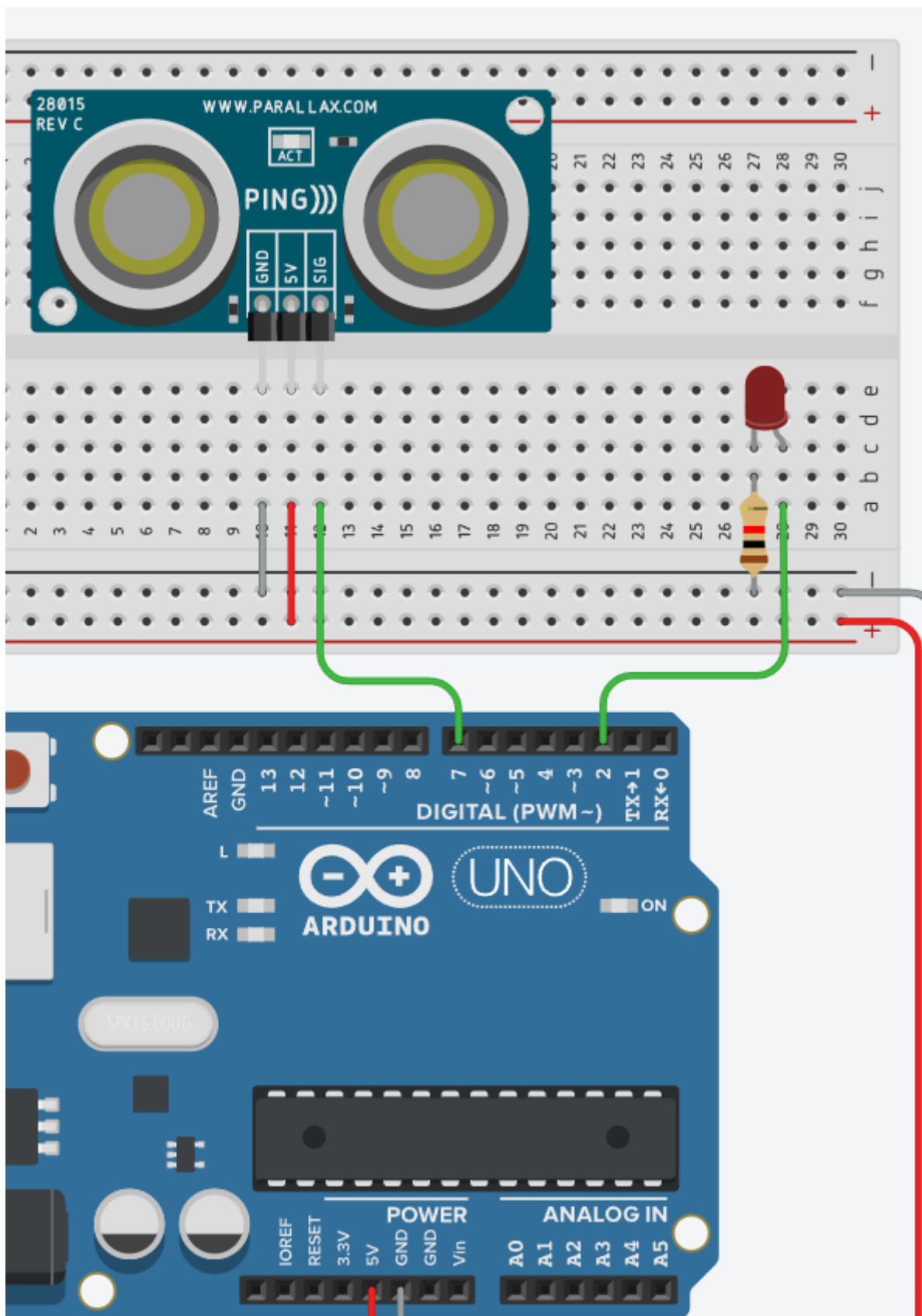


Рисунок 1.1 – Під'єднані компоненти

Після введення коду:

```
int cm;
const int distanceThreshold=100;
long readUltrasonicDistance(
    int triggerPin,
    int echoPin
){
    pinMode(triggerPin, OUTPUT);
    digitalWrite(triggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(triggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(triggerPin, LOW);
    pinMode(echoPin, INPUT);

    long duration=pulseIn(echoPin, HIGH);
    long cmDistance=duration/29/2;
    return cmDistance;
}

void setup()
{
    Serial.begin(9600);
    pinMode(2, OUTPUT);
}

void loop()
{
    cm=readUltrasonicDistance(7,7);
    Serial.print(cm);
    Serial.print("cm, ");
    if (cm>distanceThreshold){
        digitalWrite(2, LOW);
    }
    if (cm<=distanceThreshold){
        digitalWrite(2, HIGH);
    }
    delay(100);
}
```

Програма не працювала, бо не було додано символ ; після рядку `return cmDistance` у функції `readUltrasonicDistance`. Після додавання символу все запрацювало.

Після спроби прибрати першу частину функції `readUltrasonicDistance` було виявлено що без неї світлодіод горить постійно, але її значення ще не зрозуміле. Також не зрозуміло чому значення, отримане від функції `pulseIn` ділиться на 29 і на 2.

Код відформатовано та трохи спрощено.

[Посилання на проєкт.](#)

```
int distanceInCm;
const int distanceThreshold = 100;

const int ultrasonicPin = 7;
const int ledPin = 2;

long getDistanceFromSensor(int triggerPin, int echoPin)
{
    // without this part the LED is constantly lit
    pinMode(triggerPin, OUTPUT);
    digitalWrite(triggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(triggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(triggerPin, LOW);

    pinMode(echoPin, INPUT);
    long duration = pulseIn(echoPin, HIGH);
    long cmDistance = duration / 29 / 2;
    return cmDistance;
}

void setup()
{
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);
}

void loop()
{
    distanceInCm = getDistanceFromSensor(ultrasonicPin, ultrasonicPin);
    Serial.print(distanceInCm);
    Serial.print(" cm\n");
    if (distanceInCm > distanceThreshold)
    {
        digitalWrite(ledPin, LOW);
    }
    else
    {
        digitalWrite(ledPin, HIGH);
    }
    delay(100);
}
```

## **2 Приклад сервоприводу**

Було додано плату Arduino та сервопривід, його підключено до плати:

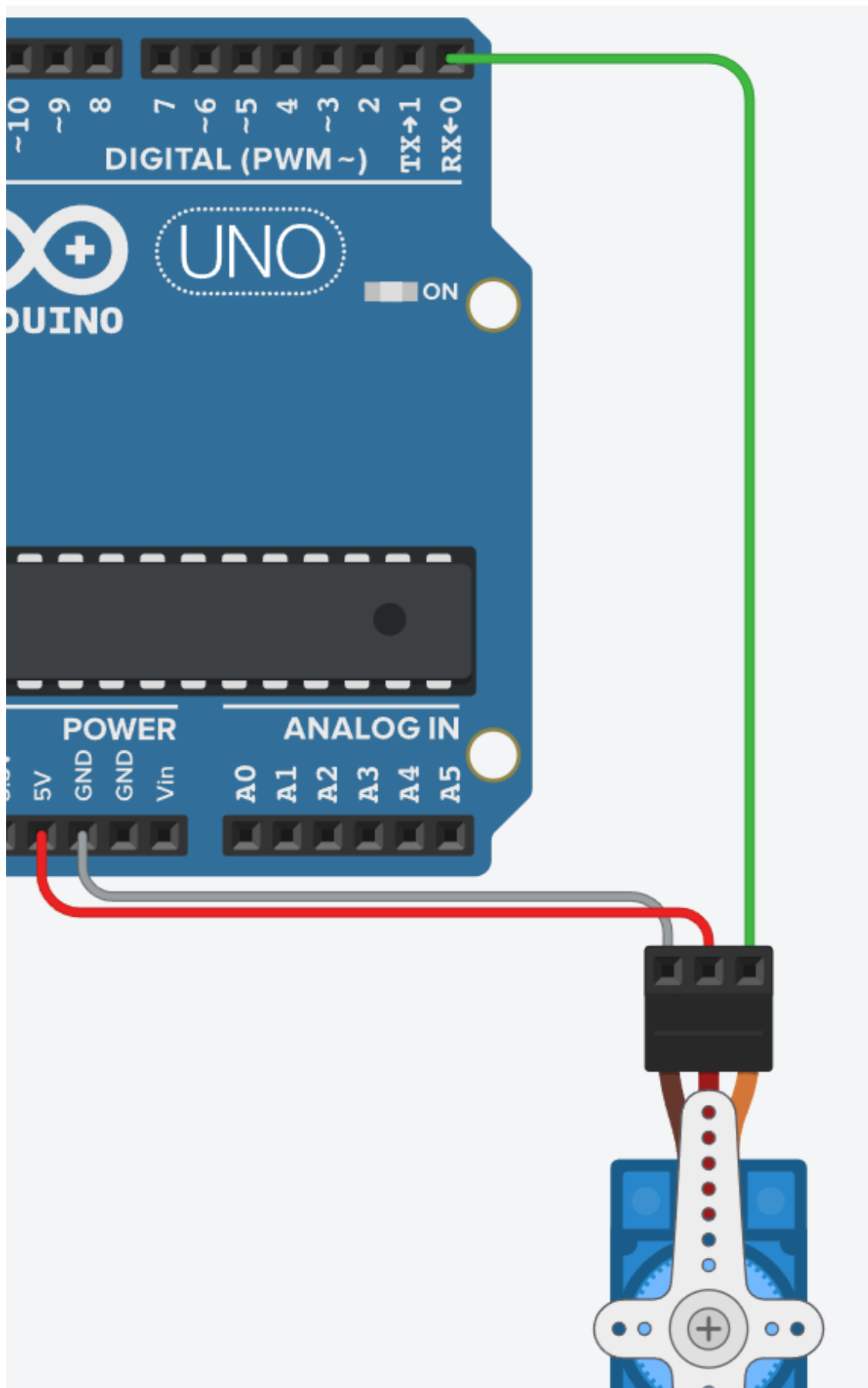


Рисунок 2.1 – Підключений сервопривід до плати



Після написання коду:

```
#include <Servo.h>
Servo servo;

bool on = false;

const int servoPin = 12;

void setup()
{
  Serial.begin(9600);
  servo.attach(servoPin);
}

void loop()
{
  if (on)
  {
    servo.write(180);
  }
  else
  {
    servo.write(-180);
    on = false;
  }
  delay(2000);
}
```

Мотор працював: при ввімкненні сервопривід обертався на 180 градусів.

Потім було додано макетоплату, до неї підключено перемикач та сервопривід:

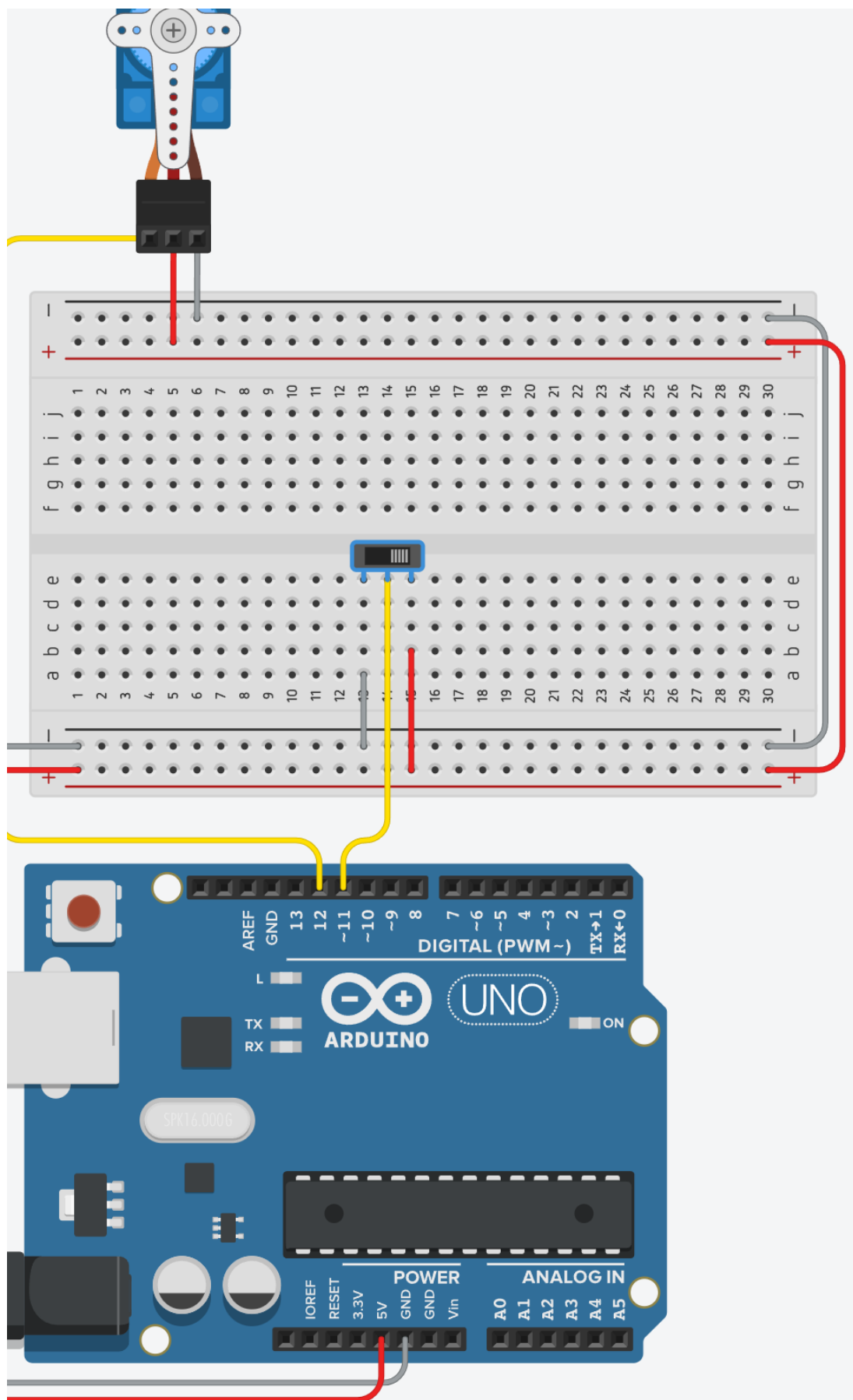


Рисунок 2.2 – Підключена макетна плата

Код було модифіковано:

```
#include <Servo.h>
Servo servo;

int switchState = 0;

const int servoPin = 12;
const int switchPin = 11;

void setup()
{
  Serial.begin(9600);
  servo.attach(servoPin);
  pinMode(switchPin, INPUT);
}

void loop()
{
  switchState = digitalRead(switchPin);
  if (switchState == HIGH)
  {
    servo.write(90);
  }
  else
  {
    servo.write(-90);
  }
  delay(12);
}
```

При запуску сервопривід крутиться на 90 градусів вперед якщо перемикач ввімкнений і на 90 градусів назад якщо вимкнений.

[Посилання на проєкт.](#)

### **З П'ять світлодіодів**

Додано макетоплату, ультразвуковий датчик, плату Arduino та світлодіоди з резисторами:

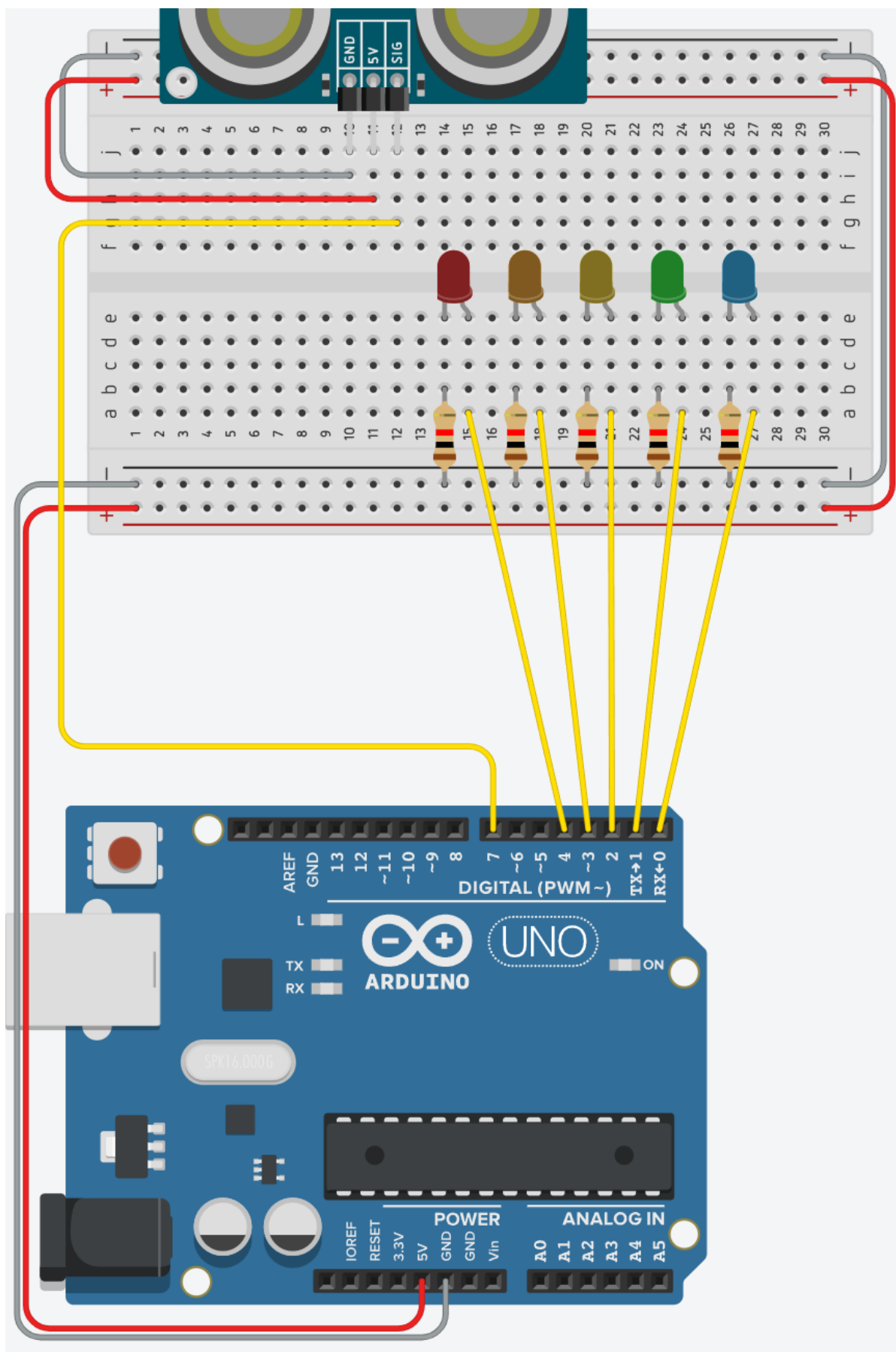


Рисунок 3.1 – Додані компоненти системи

Для початку було написано такий код:

```
const int redLedPin = 4;
const int orangeLedPin = 3;
const int yellowLedPin = 2;
const int greenLedPin = 1;
const int blueLedPin = 0;

void loop()
{
    int distance_to_object = 200;
    if (distance_to_object < 50)
    {
        digitalWrite(redLedPin, HIGH);
        digitalWrite(orangeLedPin, HIGH);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 50 && distance_to_object < 100)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, HIGH);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 100 && distance_to_object < 150)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 150 && distance_to_object < 200)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, LOW);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 200 && distance_to_object < 300)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, LOW);
        digitalWrite(greenLedPin, LOW);
        digitalWrite(blueLedPin, HIGH);
    }
    else
    {
        digitalWrite(redLedPin, LOW);
    }
}
```

```
digitalWrite(orangeLedPin, LOW);  
digitalWrite(yellowLedPin, LOW);  
digitalWrite(greenLedPin, LOW);  
digitalWrite(blueLedPin, LOW);  
}  
}
```

Через відсутність функції `setup` програма не працювала.

Після додавання функції `setup` програма працювала, але всі світлодіоди горіли постійно, не залежно від позиції об'єкта. Проблема була в тому, що у функції `getDistanceData` в рядку обрахунку дистанції замість першого знака розділення стояв знак дорівнює:

```
long cm = dur = 29 / 2;
```

Після виправлення все запрацювало.

[Посилання на проєкт.](#)

```
const int redLedPin = 4;  
const int orangeLedPin = 3;  
const int yellowLedPin = 2;  
const int greenLedPin = 1;  
const int blueLedPin = 0;  
  
const int sensorPin = 7;  
  
long getDistanceData()  
{  
    pinMode(sensorPin, OUTPUT);  
    digitalWrite(sensorPin, LOW);  
    delayMicroseconds(2);  
    digitalWrite(sensorPin, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(sensorPin, LOW);  
  
    pinMode(sensorPin, INPUT);  
    long dur = pulseIn(sensorPin, HIGH);  
    long cm = dur / 29 / 2;  
    return cm;  
}  
  
void setup()  
{  
    pinMode(redLedPin, OUTPUT);  
    pinMode(orangeLedPin, OUTPUT);  
    pinMode(yellowLedPin, OUTPUT);  
    pinMode(greenLedPin, OUTPUT);  
}
```

```

    pinMode(blueLedPin, OUTPUT);
}

void loop()
{
    long distance_to_object = getDistanceData();
    if (distance_to_object < 50)
    {
        digitalWrite(redLedPin, HIGH);
        digitalWrite(orangeLedPin, HIGH);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 50 && distance_to_object <= 100)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, HIGH);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 100 && distance_to_object <= 150)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, HIGH);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 150 && distance_to_object <= 200)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, LOW);
        digitalWrite(greenLedPin, HIGH);
        digitalWrite(blueLedPin, HIGH);
    }
    else if (distance_to_object > 200 && distance_to_object <= 300)
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, LOW);
        digitalWrite(greenLedPin, LOW);
        digitalWrite(blueLedPin, HIGH);
    }
    else
    {
        digitalWrite(redLedPin, LOW);
        digitalWrite(orangeLedPin, LOW);
        digitalWrite(yellowLedPin, LOW);
        digitalWrite(greenLedPin, LOW);
        digitalWrite(blueLedPin, LOW);
    }
    delay(1000);
}

```

## 4 Яскравий світлодіод

Додано плату Arduino, ультразвуковий датчик та світлодіод:

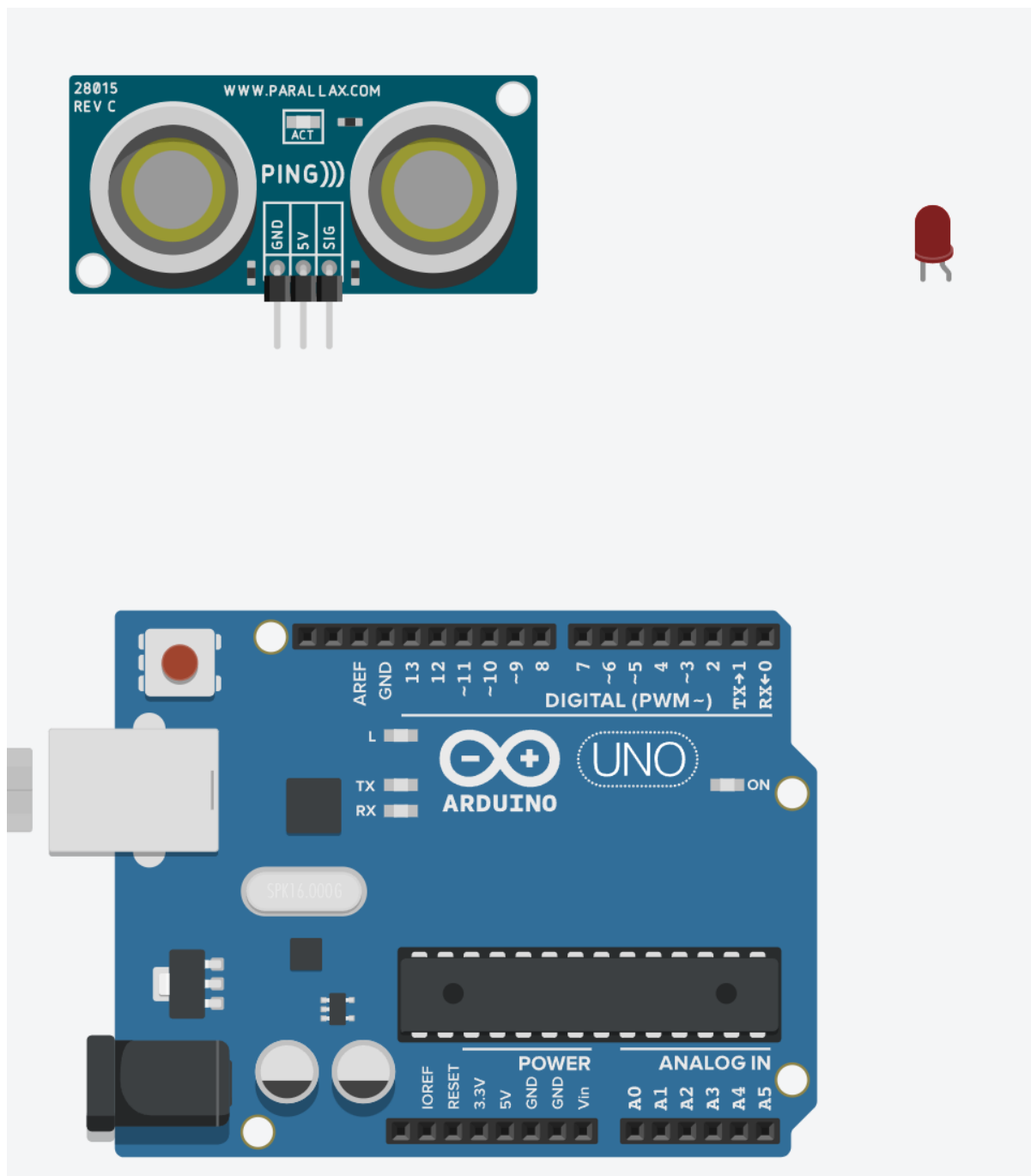


Рисунок 4.1 – Додані компоненти



Далі було знайдено такий публічний проєкт ([Control LED Brightness using Potentiometer by Wan Muhammad Faris Bin Wan Mat on Tinkercad Public](#)). В цьому проєкті автор використовує функцію `analogWrite` і значення яскравості як її другий параметр для регулювання світлодіода.

До схеми було додано макетоплату, всі компоненти підключено:

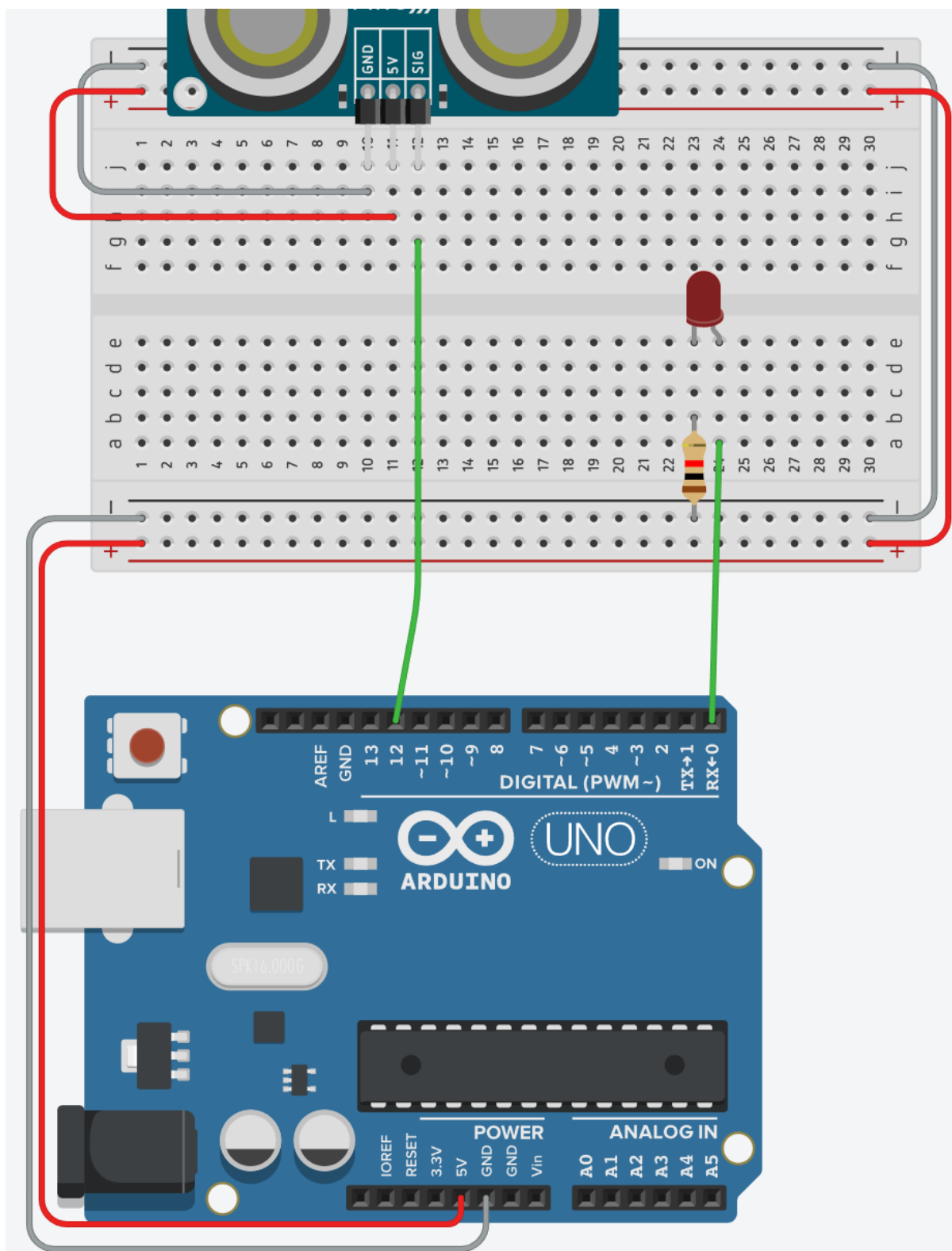


Рисунок 4.2 – Підключні компоненти

Після написання коду:

```

const int sensorPin = 12;
const int ledPin = 0;

long getDistance()
{
  pinMode(sensorPin, OUTPUT);
  digitalWrite(sensorPin, LOW);
  delayMicroseconds(2);
  digitalWrite(sensorPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(sensorPin, LOW);

  pinMode(sensorPin, INPUT);
  long dur = pulseIn(sensorPin, HIGH);
  long cm = dur / 29 / 2;
  return cm;
}

void setup()
{
  pinMode(sensorPin, OUTPUT);
}

void loop()
{
  long distance = getDistance();
  analogWrite(sensorPin, distance);
}

```

Світлодіод загорявся на початку, але яскравості не змінював.

Після перепідключення світлодіода (мінус підключено до землі, а резистор переставлено на плюс і підключено до піна) при підключенні до піна 0 діод так само горів з постійною яскравістю. Після підключення плюса до піна 3 на платі Arduino світлодіод перестав горіти.

Потім було помічено, що у функції `setup` метод `pinMode` був застосований до піна сенсору замість піна світлодіода:

```
pinMode(sensorPin, OUTPUT);
```

Після виправлення світлодіод не змінював яскравості.

Тоді було змінено функцію `getDistance` (повертає тепер значення `dur` без ділення на 29 і на 2) і як аргумент для `analogWrite` було задано отримане значення `dur` від сенсора поділене на 4, як [у прикладі](#):

```
analogWrite(sensorPin, distance/4);
```

Після змін світлодіод не змінював яскравості.

Тоді було помічено, що як перший аргумент функції `analogWrite` було введено пін сенсора.

```
analogWrite(ledPin, distance);
```

Після виправлення нічого не змінилось.

Після перепідключення світлодіода до пінa 3 він почав змінювати яскравість, але не у правильному порядку: десь на 4/5 дистанції об'єкта світлодіод горів максимально яскраво, на початку і в кінці трохи менш яскраво.

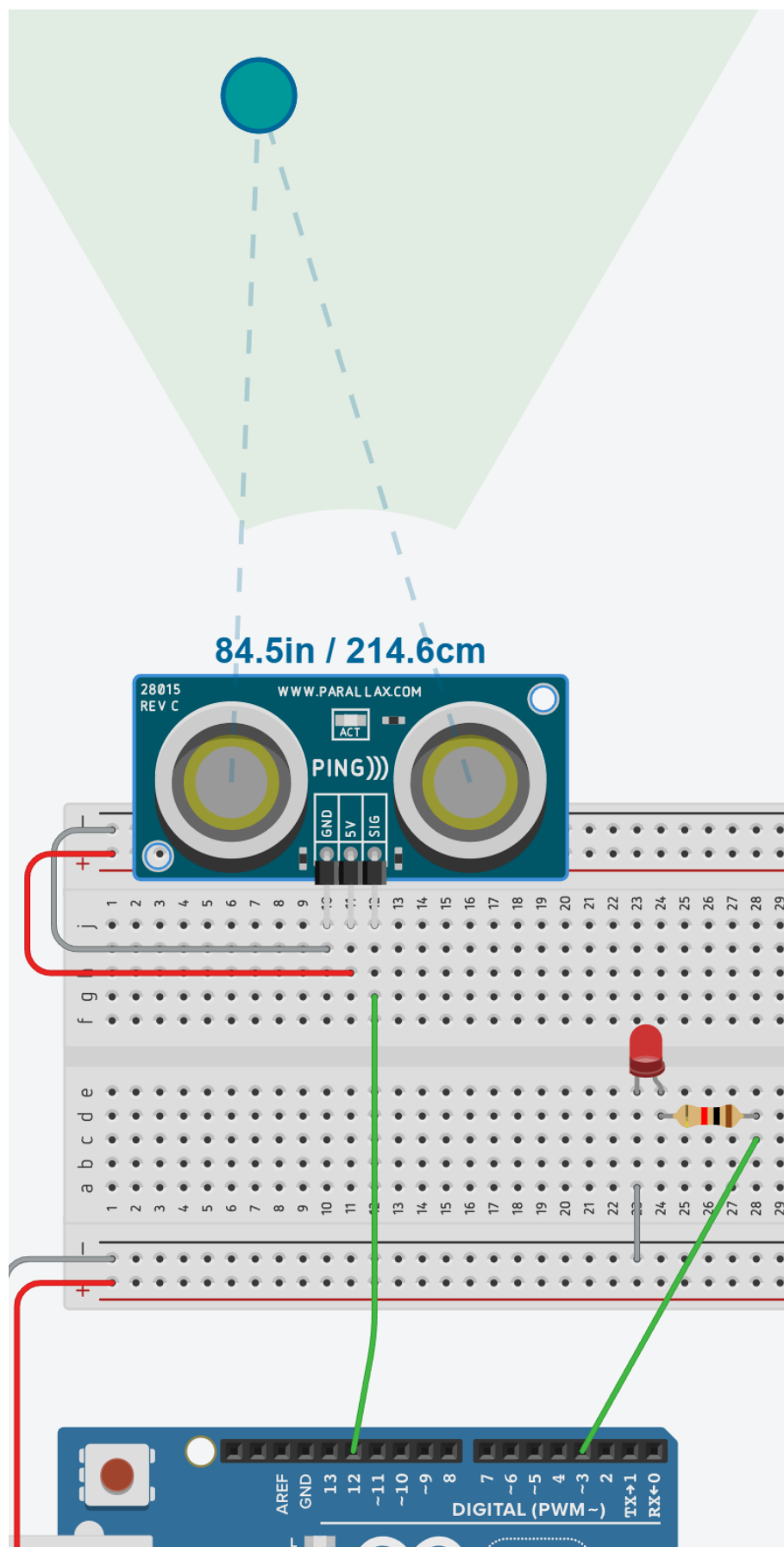


Рисунок 4.3 – Світлодіод на приблизно максимальній яскравості

Після цього звідси ([Convert a number range to another range, maintaining ratio – StackOverflow](#)) було взято і додано функцію

convertDistanceRange:

```
long convertDistanceRange(long oldValue)
{
    const long oldMin = 0;
    const long oldMax = 300;
    const long newMin = 0;
    const long newMax = 255;

    long oldRange = (oldMax - oldMin);
    long newRange = (newMax - newMin);
    long newValue = (((oldValue - oldMin) * newRange) / oldRange) +
newMin;

    return newValue;
}
```

Також при виведенні сигналу на світлодіод значення було сконвертоване через цю функцію та наведено у зворотньому вигляду (щоб при наближенні світлодіод горів більше, а не менше).

[Посилання на проєкт.](#)

```
const int sensorPin = 12;
const int ledPin = 3;

long getDistance()
{
    pinMode(sensorPin, OUTPUT);
    digitalWrite(sensorPin, LOW);
    delayMicroseconds(2);
    digitalWrite(sensorPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(sensorPin, LOW);

    pinMode(sensorPin, INPUT);
    long dur = pulseIn(sensorPin, HIGH);
    long cm = dur / 29 / 2;
    return cm;
}

long convertDistanceRange(long oldValue)
{
    const long oldMin = 0;
    const long oldMax = 300;
```

```

    const long newMin = 0;
    const long newMax = 255;

    long oldRange = (oldMax - oldMin);
    long newRange = (newMax - newMin);
    long newValue = (((oldValue - oldMin) * newRange) / oldRange) +
newMin;

    return newValue;
}

void setup()
{
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);
}

void loop()
{
    long distance = getDistance();
    Serial.print(distance);
    Serial.print(" cm\n");
    delay(500);
    if (distance<0 || distance>300){
        analogWrite(ledPin, LOW);
    } else {
        analogWrite(ledPin, -convertDistanceRange(distance));
    }
}

```

## 5 Сервопривід і сенсор

Додано ультразвуковий датчик, сервопривід, макетоплату та плату Arudiono. Все під'єднано:

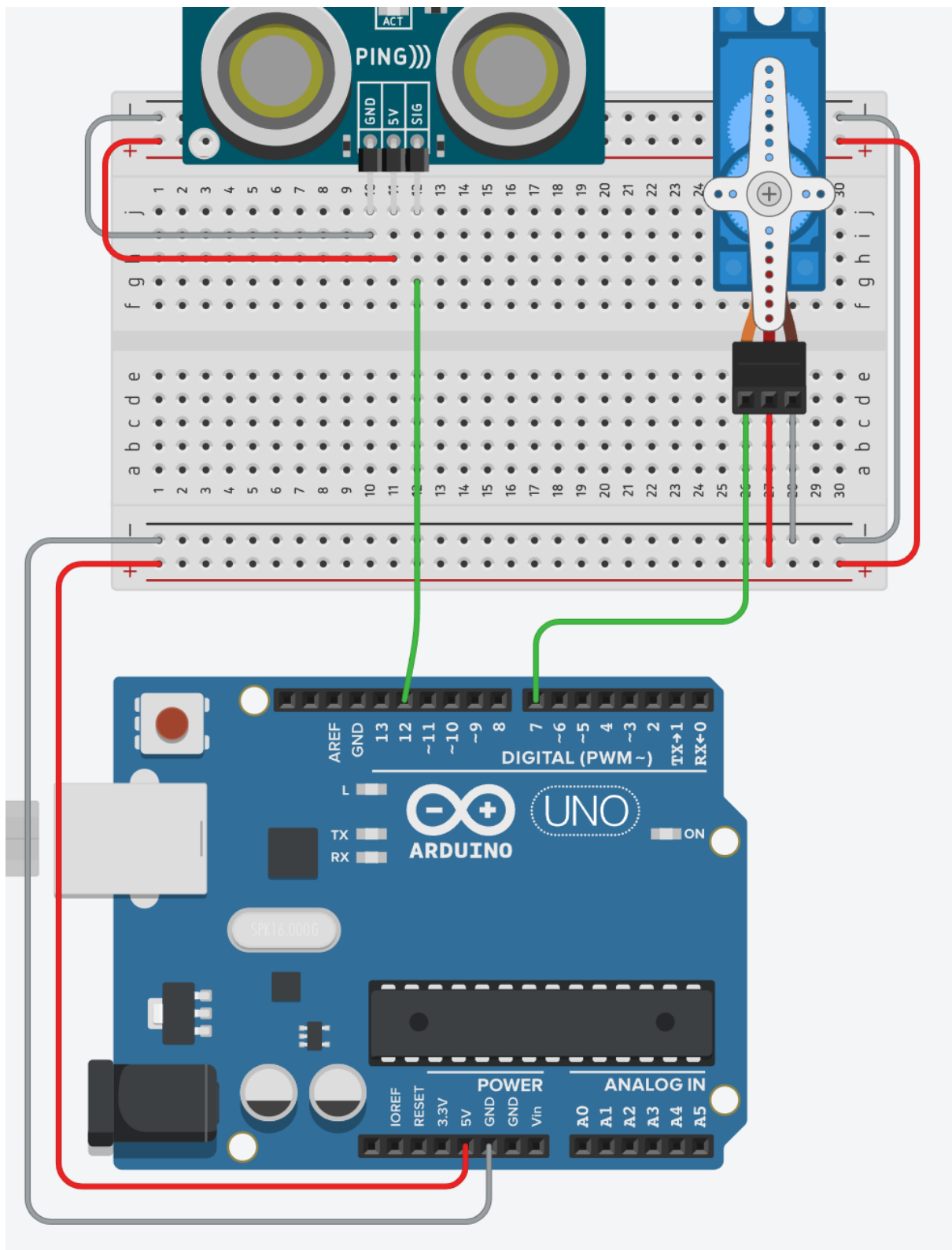


Рисунок 5.1 – Під'єднані компоненти

Після написання коду:



```

const int sensorPin = 12;
const int servoPin = 7;

long getDistance()
{
    pinMode(sensorPin, OUTPUT);
    digitalWrite(sensorPin, LOW);
    delayMicroseconds(2);
    digitalWrite(sensorPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(sensorPin, LOW);

    pinMode(sensorPin, INPUT);
    long dur = pulseIn(sensorPin, HIGH);
    long cm = dur / 29 / 2;
    return cm;
}

void setup()
{
    pinMode(servoPin, OUTPUT);
}

void loop()
{
    long distance = getDistance();
    int degrees = 90;
    if (distance < 100)
    {
        degrees = 0;
    }
    else if (distance >= 100)
    {
        degrees = 180;
    }
    digitalWrite(servoPin, degrees);
    delay(1000);
}

```

**Програма не працювала.**

**Після додавання бібліотеки Servo.h та написання правильних методів:**

```

#include <Servo.h>
Servo servo;

const int sensorPin = 12;
const int servoPin = 7;

long getDistance()
{
    pinMode(sensorPin, OUTPUT);
    digitalWrite(sensorPin, LOW);

```

```

    delayMicroseconds(2);
    digitalWrite(sensorPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(sensorPin, LOW);

    pinMode(sensorPin, INPUT);
    long dur = pulseIn(sensorPin, HIGH);
    long cm = dur / 29 / 2;
    return cm;
}

void setup()
{
    pinMode(servoPin, OUTPUT);
    servo.attach(servoPin);
}

void loop()
{
    long distance = getDistance();
    int degrees = 90;
    if (distance < 100)
    {
        degrees = 0;
    }
    else if (distance >= 100 && distance <= 300)
    {
        degrees = 180;
    }
    else
    {
        degrees = 90;
    }
    servo.write(degrees);
    delay(1000);
}

```

Все запрацювало.

[Посилання на проєкт.](#)

## ВИСНОВКИ

В результаті виконання другої роботи було ознайомлено з пристроями «Ультразвуковий датчик вимірювання відстані» та «Сервопривід» в середовищі Tinkercad.